**UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA**

**FACULTAD DE INGENIERÍA EN SISTEMAS**

**CURSO: SISTEMAS OPERATIVOS**

**SECCIÓN "A"**

**CATEDRÁTICO: ING. EDWIN CÓRDOVA**

# DOCUMENTACIÓN TÉCNICA Y UN CUADRO COMPARATIVO ANALIZANDO EL ALGORITMO QUE DEMOSTRÓ MAYOR EFICIENCIA EN EL PROGRAMA DEL PROYECTO FINAL DE SISTEMAS OPERATIVOS


| NOMBRE: | CARNÉ: |
|---|---|
| DANIEL ANDRÉ SANDOVAL CHÁVEZ | 0908-24-5040 |
| DUSTIN STEVE SANTOS MONTERROSO | 0908-24-3397 |
| ERICK NEFTALÍ CAMARGO JIMÉNEZ | 0908-23-9943 |
| LUCIA VALERIA MOLLINEDO HERNÁNDEZ | 0908-24-879 |
| ALESSANDRA MARINELI CONTRERAS GÓMEZ | 0908-24-5044 |


**ESCUINTLA, OCTUBRE 2026**

# Índice


1. Resumen ..... 1
2. Documentación técnica ..... 1
   2.1 Descripción general ..... 1
   2.2 Arquitectura ..... 1
   2.3 Modelo de datos ..... 2
   2.4 Contrato de los planificadores ..... 2
   2.5 Algoritmos ..... 3
      FCFS (no apropiativo) ..... 3
      SJF, SRTF y Prioridades (selección por clave) ..... 3
      Round Robin (apropiativo) ..... 3
   2.6 Métricas ..... 3
   2.7 Interfaz gráfica y animación ..... 4
   2.8 Decisiones de diseño y casos borde ..... 4
   2.9 Pruebas automáticas ..... 4
3. Metodología de la comparación ..... 5
4. Casos de prueba y resultados ..... 5
   4.1 Caso 1: ejemplo clásico ..... 5
   4.2 Caso 2: proceso largo primero ..... 6
   4.3 Caso 3: llegadas dispersas con CPU ociosa ..... 7
5. Cuadro comparativo general ..... 9
   Sensibilidad de Round Robin al quantum ..... 10
6. Análisis y conclusión ..... 10

# Documentación técnica y análisis comparativo

Simulador de Planificador de Procesos (CPU Scheduler)
17 de octubre de 2026 | Hecho por: Daniel André Sandoval Chávez

## 1. Resumen


> En los tres casos de prueba, **SRTF (SJF apropiativo)** fue el algoritmo más eficiente: obtuvo la menor espera promedio y el menor retorno promedio en todos los casos (en el Caso 2 comparte el primer lugar con SJF y con Prioridades, y en el Caso 3 con Prioridad apropiativo). Su espera promedio media fue **4.00** unidades de tiempo, frente a 10.03 de FCFS y 7.67 de Round Robin con quantum 3.

La utilización de la CPU fue idéntica para todos los algoritmos dentro de cada caso, porque ninguno deja la CPU libre mientras hay procesos listos; por eso la diferencia de eficiencia se ve en los tiempos de espera y de retorno. La ventaja de SRTF tiene un costo: ejecuta más turnos de CPU (16 en total frente a 13 de FCFS) y exige conocer de antemano la ráfaga de cada proceso. Las secciones 2 y 3 documentan el simulador y el método; las secciones 4 a 6 presentan los casos, el cuadro comparativo y el análisis.


## 2. Documentación técnica

### 2.1 Descripción general

El simulador es una aplicación de escritorio escrita en Python 3.9 o superior que usa solo la librería estándar (tkinter para la interfaz). Ejecuta una misma lista de procesos con seis algoritmos de planificación de CPU: FCFS, SJF no apropiativo, SRTF (SJF apropiativo), Round Robin con quantum ingresado por el usuario y, como algoritmo extra, Prioridades en sus variantes apropiativa y no apropiativa. Para cada ejecución muestra un diagrama de Gantt animado y calcula el tiempo de espera y el de retorno por proceso y en promedio.

### 2.2 Arquitectura

El código se divide en módulos con una sola responsabilidad. Solo `gui.py` depende de tkinter: los algoritmos y las métricas son funciones puras, por lo que se prueban sin interfaz y la interfaz puede cambiar sin reescribirlos. La tabla describe cada archivo y la figura muestra cómo se relacionan.

| Archivo | Responsabilidad |
|---|---|
| `models.py` | Clase Proceso (bloque de control de proceso simplificado). |
| `schedulers.py` | Algoritmos FCFS, SJF/SRTF, Round Robin y Prioridades. |
| `metrics.py` | Cálculo de retorno, espera, promedios y utilización de CPU. |
| `gui.py` | Ventana principal, animación del Gantt, tabla de métricas y comparación. |
| `main.py` | Punto de entrada: crea y ejecuta la ventana. |
| `tests.py` | Diez pruebas automáticas con resultados calculados a mano. |
| `casos_de_prueba.py` | Ejecuta los tres casos de este documento y reproduce el cuadro comparativo. |

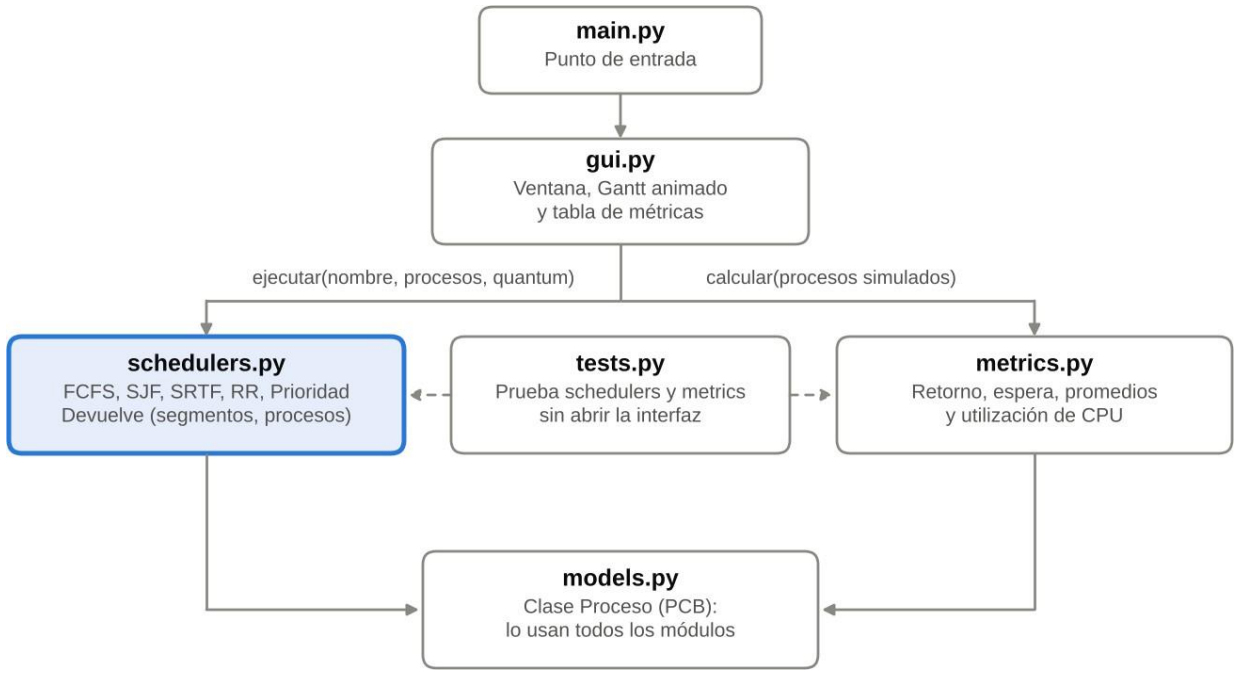



**Figura 1.** Módulos del proyecto y dependencias entre ellos. Se resalta schedulers.py, que concentra la lógica de planificación. Línea discontinua: tests.py importa schedulers y metrics.

## 2.3 Modelo de datos

Cada proceso es una instancia de la clase `Proceso` (`dataclass`):

| Atributo | Origen | Descripción |
|---|---|---|
| `pid` | Usuario | Identificador del proceso (texto). No puede repetirse ni llamarse IDLE. |
| `llegada` | Usuario | Tiempo de arribo (entero, mayor o igual a 0). |
| `rafaga` | Usuario | Tiempo de CPU requerido (entero, mayor que 0). |
| `prioridad` | Usuario | Opcional. A menor número, mayor prioridad. Por defecto 0. |
| `restante` | Simulación | Tiempo de CPU que aún le falta; empieza igual a la ráfaga. |
| `finalizacion` | Simulación | Instante en que el proceso termina. |

El método `copia()` devuelve un proceso nuevo con solo los datos de entrada. Cada algoritmo trabaja sobre copias, así la misma lista original se ejecuta con todos los algoritmos sin que uno afecte a otro.

## 2.4 Contrato de los planificadores

Todos los algoritmos reciben la lista de procesos y devuelven la tupla `(segmentos, procesos)`. Un segmento `(pid, inicio, fin)` indica que `pid` usa la CPU en el intervalo [inicio, fin); los intervalos sin procesos listos se registran con el pid reservado `IDLE`. La lista de segmentos alimenta el diagrama de Gantt, y los procesos con su `finalizacion` alimentan las métricas. La función `_fusionar()` une segmentos contiguos del mismo proceso, y `ejecutar(nombre, procesos, quantum)` despacha por nombre y es la única función que llama la interfaz.

## 2.5 Algoritmos

### FCFS (no apropiativo)

Ordena por llegada (el orden estable respeta el orden de ingreso en los empates) y ejecuta cada proceso completo.

```
t = 0
para cada proceso p ordenado por llegada:
    si t < p.llegada: registrar IDLE(t, p.llegada); t = p.llegada registrar
    (p.pid, t, t + p.rafaga)
    t = t + p.rafaga; p.finalizacion = t
```

### SJF, SRTF y Prioridades (selección por clave)

Una sola función genérica elige, entre los procesos ya llegados, el de menor clave: el tiempo restante en SJF y SRTF, o la prioridad en Prioridades. El desempate es la menor llegada y después el orden de ingreso. En modo no apropiativo el elegido corre hasta terminar; en modo apropiativo corre hasta terminar o hasta la próxima llegada, momento en que se vuelve a decidir.

```
mientras queden procesos pendientes:
    listos = {p : p.restante > 0 y p.llegada <= t}
    si listos está vacío: registrar IDLE hasta la próxima llegada; continuar
    elegido = min(listos, por (clave, llegada))
    si no apropiativo: duracion = elegido.restante
    si apropiativo:     duracion = min(elegido.restante, proxima_llegada - t)
    registrar (elegido.pid, t, t + duracion)
    t += duracion; elegido.restante -= duracion
    si elegido.restante == 0: elegido.finalizacion = t
```

En modo apropiativo solo se reevalúa la decisión en cada llegada: entre llegadas, el proceso en ejecución solo se acerca a terminar y los demás no cambian, así que sigue siendo el mejor. El resultado es el mismo que reevaluar en cada unidad de tiempo, con menos pasos.

### Round Robin (apropiativo)

Usa dos colas: `por_llegar` (procesos que aún no llegan) y `cola` (listos). La función interna `admitir(hasta)` mueve a la cola los procesos con llegada menor o igual a `hasta`. Si el quantum es menor o igual a 0 se lanza `ValueError`.

```
mientras queden procesos sin terminar: admitir(t)
    si cola está vacía: registrar IDLE hasta la próxima llegada; continuar p =
    cola.sacar_primero()
    duracion = min(quantum, p.restante) registrar
    (p.pid, t, t + duracion)
    t += duracion; p.restante -= duracion
    admitir(t)                      # los que llegaron durante la ejecución entran primero si
    p.restante > 0: cola.agregar(p)
    si no: p.finalizacion = t
```

## 2.6 Métricas

`metrics.calcular(procesos)` calcula las mismas métricas para cualquier algoritmo y guarda también el cálculo desarrollado como texto (por ejemplo `17 - 0 = 17`) para mostrarlo en pantalla.

| Métrica | Fórmula |
|---|---|
| Tiempo de retorno | Finalización - Llegada |
| Tiempo de espera | Retorno - Ráfaga |
| Promedios | Suma de los valores de todos los procesos dividida entre la cantidad de procesos |
| Utilización de CPU | (Suma de las ráfagas / Tiempo final) x 100 |

## 2.7 Interfaz gráfica y animación

La clase `App(tk.Tk)` organiza la ventana en cuatro paneles: ingreso de procesos, algoritmo y simulación, diagrama de Gantt y métricas. Al pulsar Simular se validan los datos, se ejecuta el algoritmo elegido, el resultado se descompone en unidades de tiempo y `_animar()` dibuja una unidad por llamada, reprogramándose con `after()` de tkinter para no congelar la ventana. Al agotar las unidades `_finalizar()` llena la tabla de métricas con el cálculo desarrollado y los promedios. El botón Saltar animación dibuja de golpe lo que falta, y Comparar todos ejecuta los seis algoritmos con la misma lista y muestra sus promedios en una ventana aparte.

## 2.8 Decisiones de diseño y casos borde

| Caso o decisión | Tratamiento |
|---|---|
| Ningún proceso ha llegado (CPU libre) | Se registra un segmento IDLE hasta la próxima llegada, en todos los algoritmos. |
| Empate en SJF, SRTF o Prioridad | Gana la menor llegada; si persiste, el orden de ingreso. |
| Llega un proceso justo cuando termina un quantum (Round Robin) | El que llega entra a la cola antes que el interrumpido (convención documentada). |
| Quantum menor o igual a 0 | Round Robin lanza ValueError; la interfaz lo valida antes de llamarlo. |
| Ráfaga 0, llegada negativa o PID repetido | La interfaz rechaza el ingreso con un mensaje de error. |
| Procesos ingresados fuera de orden de llegada | Cada algoritmo los ordena internamente. |
| Separar lógica e interfaz | Permite probar los algoritmos sin ventana y cambiar la interfaz sin tocar la lógica. |
| Un solo selector genérico para SJF, SRTF y Prioridades | Solo cambian la clave y si es apropiativo; evita código duplicado. |

## 2.9 Pruebas automáticas

El archivo `tests.py` contiene diez pruebas que comparan la salida de los algoritmos con resultados calculados a mano: FCFS, SJF, SRTF, Round Robin (incluida la convención de empates), CPU ociosa, desempates, Prioridades, que la lista original no se modifique y las fórmulas de las métricas. Se ejecutan con `python tests.py` y no requieren librerías externas.

# 3. Metodología de la comparación

Cada caso de prueba es una lista de procesos que se ejecutó con los seis algoritmos usando el código real del proyecto. Round Robin usó quantum 3 en los tres casos. En Prioridades, un número menor significa mayor prioridad. Para cada ejecución se registraron estas métricas:

| Métrica | Qué indica | Cómo se obtiene |
|---|---|---|
| Espera promedio | Tiempo medio que los procesos pasan listos sin usar la CPU. Menor es mejor. | metrics.calcular() |
| Retorno promedio | Tiempo medio desde que un proceso llega hasta que termina. Menor es mejor. | metrics.calcular() |
| Utilización de CPU | Porcentaje del tiempo total en que la CPU trabajó. | metrics.calcular() |
| Turnos de CPU | Cantidad de tramos de ejecución en el Gantt, sin contar IDLE. Más turnos implican más cambios de proceso. | Contados en los segmentos |
| Respuesta máxima | Mayor tiempo que algún proceso espera para recibir la CPU por primera vez. | Calculada a partir del Gantt |

**Criterio de eficiencia.** Se considera más eficiente el algoritmo con menor espera promedio. El retorno promedio sigue el mismo orden, porque el retorno de cada proceso es su espera más su ráfaga y la suma de ráfagas es la misma para todos los algoritmos de una misma lista. Los turnos y la respuesta máxima se usan para matizar el resultado, no para decidirlo.

**Reproducción.** Los resultados se pueden reproducir de dos formas:

- Ejecute `python casos_de_prueba.py` desde la carpeta del proyecto: imprime las tablas de los tres casos con los mismos valores de este documento.
- O abra la aplicación con `python main.py`, ingrese los procesos de la tabla de entrada de cada caso (sección 4), elija el algoritmo y pulse Simular, o use Comparar todos con quantum 3.

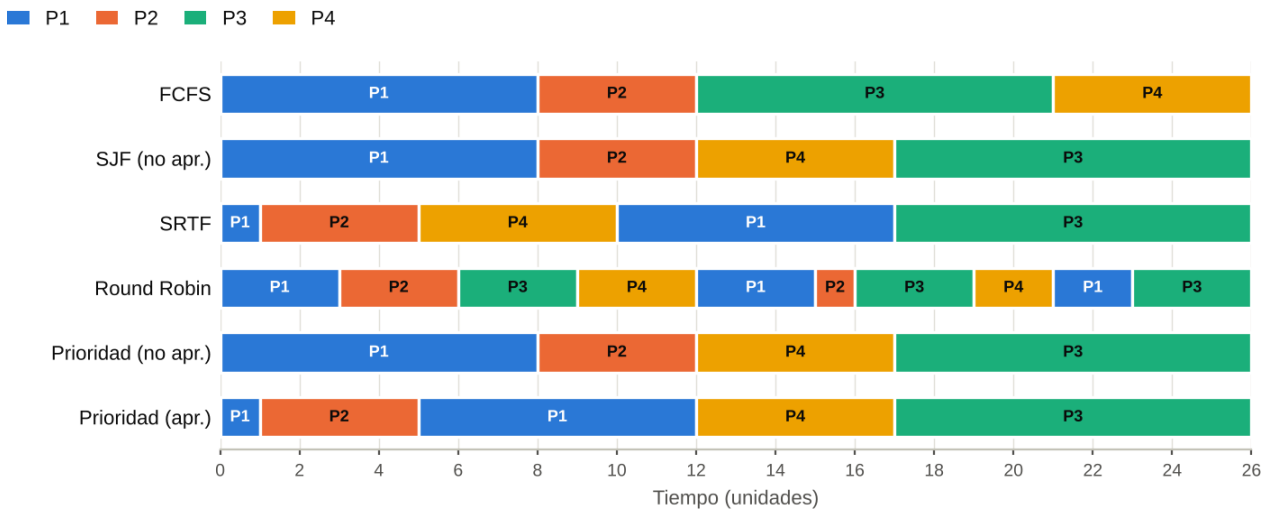
# 4. Casos de prueba y resultados

## 4.1 Caso 1: ejemplo clásico

Llegadas escalonadas y ráfagas de distinto tamaño. Es el ejemplo usado en las pruebas automáticas. Round Robin con quantum 3.

| PID | Llegada | Ráfaga | Prioridad |
|---|---|---|---|
| P1 | 0 | 8 | 2 |
| P2 | 1 | 4 | 1 |
| P3 | 2 | 9 | 3 |
| P4 | 3 | 5 | 2 |

| Algoritmo | Espera prom. | Retorno prom. | Utilización | Turnos | Resp. máx. |
|---|---|---|---|---|---|
| FCFS | 8.75 | 15.25 | 100.0 % | 4 | 18 |
| SJF (no apropiativo) | 7.75 | 14.25 | 100.0 % | 4 | 15 |
| **SRTF (SJF apropiativo)** | **6.50** | **13.00** | **100.0 %** | **5** | **15** |
| Round Robin | 13.50 | 20.00 | 100.0 % | 10 | 6 |
| Prioridad (no apropiativo) | 7.75 | 14.25 | 100.0 % | 4 | 15 |
| Prioridad (apropiativo) | 7.00 | 13.50 | 100.0 % | 5 | 15 |



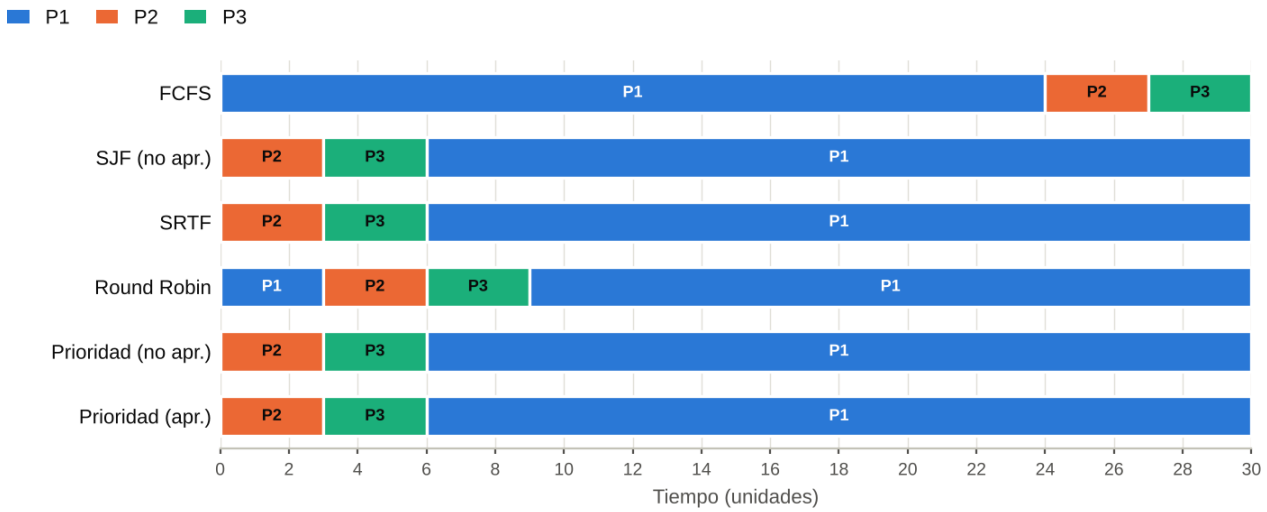

**Lectura del caso.** SRTF logra la menor espera (6.50): cuando llega P2 (ráfaga 4) le quita la CPU a P1 en lugar de esperar a que este termine. FCFS (8.75) obliga a P2, P3 y P4 a esperar todo P1. Round Robin reparte la CPU en 10 turnos y termina todos los procesos más tarde (espera 13.50), pero ninguno espera más de 6 unidades para recibir la CPU por primera vez; en SRTF, P3 espera 15.

## 4.2 Caso 2: proceso largo primero

Todos llegan en t = 0 y el proceso más largo está primero en la lista (efecto convoy en FCFS). Round Robin con quantum 3.

| PID | Llegada | Ráfaga | Prioridad |
|---|---|---|---|
| P1 | 0 | 24 | 3 |
| P2 | 0 | 3 | 1 |
| P3 | 0 | 3 | 2 |

| Algoritmo | Espera prom. | Retorno prom. | Utilización | Turnos | Resp. máx. |
|---|---|---|---|---|---|
| FCFS | 17.00 | 27.00 | 100.0 % | 3 | 27 |
| **SJF (no apropiativo)** | **3.00** | **13.00** | **100.0 %** | **3** | **6** |
| **SRTF (SJF apropiativo)** | **3.00** | **13.00** | **100.0 %** | **3** | **6** |
| Round Robin | 5.00 | 15.00 | 100.0 % | 4 | 6 |
| **Prioridad (no apropiativo)** | **3.00** | **13.00** | **100.0 %** | **3** | **6** |
| **Prioridad (apropiativo)** | **3.00** | **13.00** | **100.0 %** | **3** | **6** |




**Lectura del caso.** Con el proceso largo primero, FCFS hace esperar a P2 y P3 durante 24 y 27 unidades (espera 17.00). SJF, SRTF y Prioridades atienden antes los procesos cortos y la espera baja a 3.00. Como todos llegan en t = 0, nadie puede interrumpir a nadie y SRTF produce el mismo Gantt que SJF. Prioridades coincide porque las prioridades asignadas (P2 = 1, P3 = 2, P1 = 3) ordenan a los procesos igual que sus ráfagas.

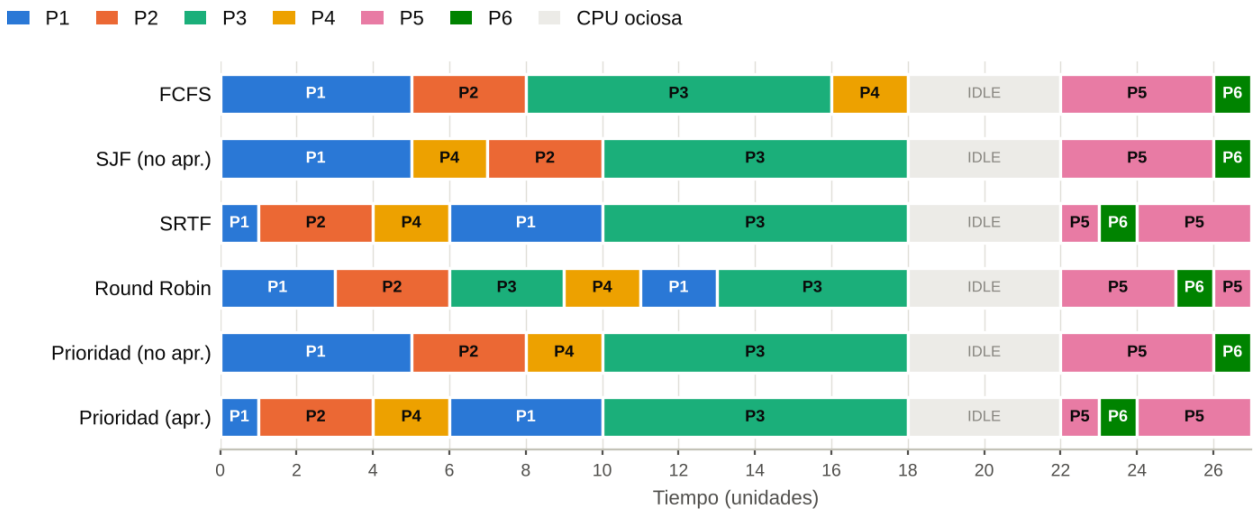
## 4.3 Caso 3: llegadas dispersas con CPU ociosa

Seis procesos en dos grupos separados por un intervalo sin trabajo (la CPU queda ociosa entre t = 18 y t = 22). Round Robin con quantum 3.

| PID | Llegada | Ráfaga | Prioridad |
|---|---|---|---|
| P1 | 0 | 5 | 2 |
| P2 | 1 | 3 | 1 |
| P3 | 2 | 8 | 3 |
| P4 | 3 | 2 | 1 |
| P5 | 22 | 4 | 2 |
| P6 | 23 | 1 | 1 |

***Tabla 10.*** *Procesos del Caso 3.*

| Algoritmo | Espera prom. | Retorno prom. | Utilización | Turnos | Resp. máx. |
|---|---|---|---|---|---|
| FCFS | 4.33 | 8.17 | 85.2 % | 6 | 13 |
| SJF (no apropiativo) | 3.17 | 7.00 | 85.2 % | 6 | 8 |
| **SRTF (SJF apropiativo)** | **2.50** | **6.33** | **85.2 %** | **8** | **8** |
| Round Robin | 4.50 | 8.33 | 85.2 % | 9 | 6 |
| Prioridad (no apropiativo) | 3.33 | 7.17 | 85.2 % | 6 | 8 |
| **Prioridad (apropiativo)** | **2.50** | **6.33** | **85.2 %** | **8** | **8** |

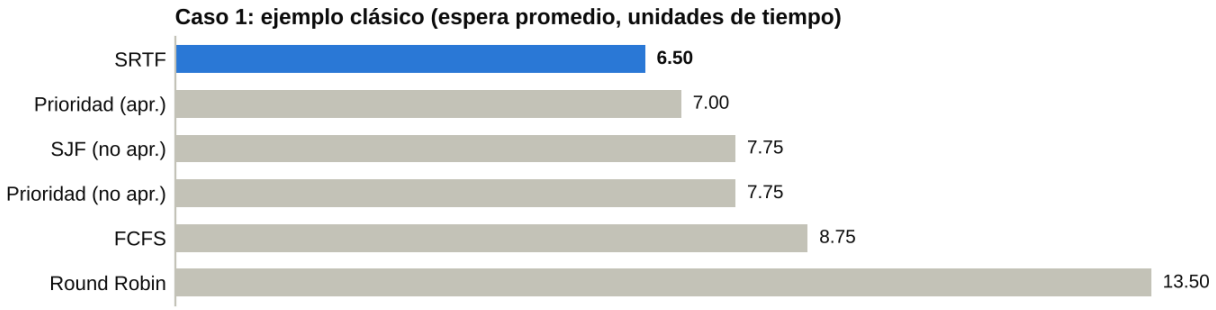


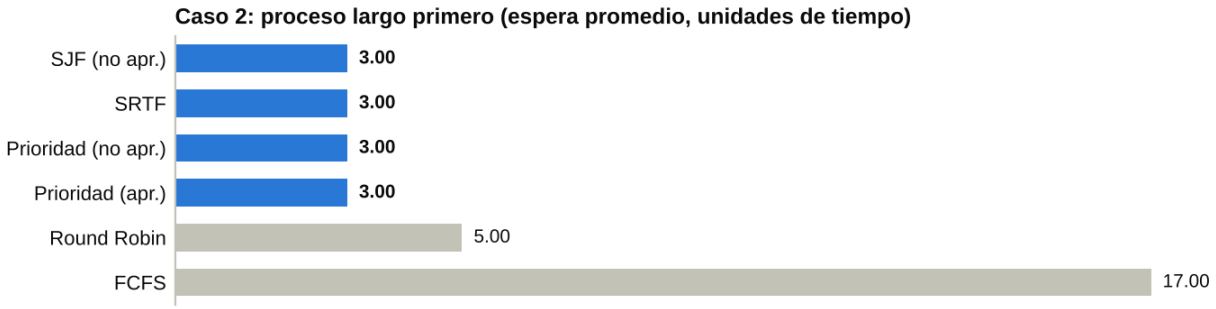

**Lectura del caso.** Entre t = 18 y t = 22 no hay procesos listos y la CPU queda ociosa en todos los algoritmos, por eso la utilización es 85.2 % (23 de 27 unidades) sin importar el algoritmo. SRTF (2.50) mejora a SJF no apropiativo (3.17) porque interrumpe a P1 cuando llega P2 (t = 1) y a P5 cuando llega P6 (t = 23). Prioridad apropiativo empata con SRTF porque las prioridades asignadas (1 para P2, P4 y P6) favorecen a los mismos procesos cortos; con otras prioridades el resultado sería distinto.

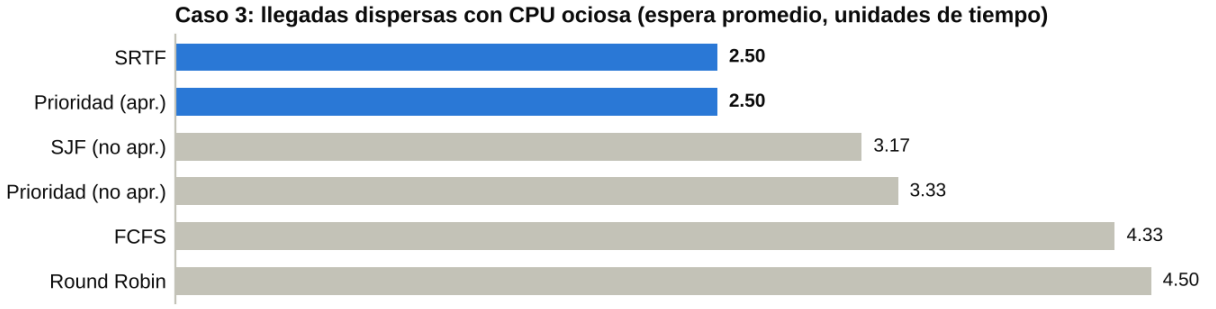
# 5. Cuadro comparativo general

La siguiente tabla resume las tres pruebas. Las celdas resaltadas marcan la menor espera de cada caso y la fila resaltada, el algoritmo con menor espera media.

| Algoritmo | Espera Caso 1 | Espera Caso 2 | Espera Caso 3 | Espera media | Retorno medio | Turnos (suma) | Menor espera en |
|---|---|---|---|---|---|---|---|
| **SRTF (SJF apropiativo)** | **6.50** | **3.00** | **2.50** | **4.00** | **10.78** | **16** | **3 de 3** |
| Prioridad (apropiativo) | 7.00 | **3.00** | **2.50** | 4.17 | 10.94 | 16 | 2 de 3 |
| SJF (no apropiativo) | 7.75 | **3.00** | 3.17 | 4.64 | 11.42 | 13 | 1 de 3 |
| Prioridad (no apropiativo) | 7.75 | **3.00** | 3.33 | 4.69 | 11.47 | 13 | 1 de 3 |
| Round Robin | 13.50 | 5.00 | 4.50 | 7.67 | 14.44 | 23 | 0 de 3 |
| FCFS | 8.75 | 17.00 | 4.33 | 10.03 | 16.81 | 13 | 0 de 3 |



### Sensibilidad de Round Robin al quantum

Como Round Robin quedó detrás de SRTF, se comprobó si un quantum distinto cambiaba esa conclusión. Se ejecutó con quantum de 1 a 8 en cada caso.

| Quantum | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| Caso 1 | 12.50 | 12.75 | 13.50 | 11.75 | 11.00 | 12.00 | 13.00 | **9.75** |
| Caso 2 | 5.67 | 6.33 | **5.00** | 5.67 | 6.33 | 7.00 | 7.67 | 8.33 |
| Caso 3 | 4.50 | 5.17 | 4.50 | 5.17 | **4.17** | 4.33 | 4.50 | 4.33 |

No existe un quantum que sea el mejor en todos los casos (el mejor fue 8, 3 y 5 en los casos 1, 2 y 3). Aun con su mejor quantum, Round Robin queda por encima de SRTF en los tres casos (9.75 frente a 6.50, 5.00 frente a 3.00 y 4.17 frente a 2.50).

# 6. Análisis y conclusión

**El algoritmo más eficiente fue SRTF.** Tuvo la menor espera promedio en los tres casos (3 de 3) y la menor espera media (4.00), seguido de Prioridad apropiativo (4.17, que depende de las prioridades asignadas) y de SJF no apropiativo (4.64). Esto es coherente con lo conocido en teoría: cuando se conocen las ráfagas, atender siempre al proceso al que menos le falta minimiza la espera promedio, porque reduce cuanto antes la cantidad de procesos que siguen esperando.

Lo que muestran los resultados sobre cada algoritmo:

- **FCFS** es sensible al orden de llegada: con un proceso largo primero (Caso 2) su espera fue 17.00, casi seis veces la de SRTF (3.00). Es el algoritmo con más espera media (10.03).
- **SJF no apropiativo** queda cerca de SRTF y usa menos turnos. Cuando nadie puede interrumpir, como en el Caso 2, ambos producen el mismo resultado.
- **Round Robin** fue peor que SRTF en espera en los tres casos y peor que FCFS en los casos 1 y 3, pero acota cuánto espera un proceso para recibir la CPU por primera vez (respuesta máxima de 6 frente a 15 de SRTF en el Caso 1). Es la opción adecuada cuando importa que todos avancen, como en un sistema interactivo.
- **Prioridades** no mide eficiencia por sí sola: su resultado depende de las prioridades asignadas. Sus empates con SRTF en los casos 2 y 3 se deben a que las prioridades elegidas favorecen a los procesos cortos; no son una ventaja propia del algoritmo.

**Costos y limitaciones de la conclusión.**

- SRTF ejecutó más turnos que FCFS y SJF (16 frente a 13 en total) porque interrumpe procesos. En un sistema real cada interrupción tiene un costo de cambio de contexto que este simulador no mide.
- Necesita conocer de antemano la ráfaga de cada proceso, algo que un sistema real solo puede estimar, y puede dejar esperando a los procesos largos (en el Caso 1, P3 espera 15 unidades antes de ejecutarse).
- Son tres casos de prueba: el resultado confirma la teoría en estos casos, pero no sustituye un análisis con cargas más variadas.

**Conclusión.** Si el objetivo es minimizar el tiempo de espera y de retorno promedio y se conocen las ráfagas, SRTF es la mejor opción de las probadas. Si el objetivo es repartir la CPU de forma justa y acotar el tiempo de primera respuesta, Round Robin es preferible aunque aumente la espera promedio.